Neural Causal Fairness Model Notebook

Hanita Haller
Columbia Unversity
hkh2122@columbia.edu

Abstract

This paper introduces a Python-based notebook designed to facilitate causal fairness analysis through the use of Neural Causal Models (NCMs). The tool enables users to define and train a structural causal model, project it onto a Standard Fairness Model (SFM), and perform fairness tasks such as bias detection, quantification, and fair prediction—all with minimal code and no need for retraining when modifying the fairness task. This modular approach separates model training from downstream analysis, allowing users to efficiently explore multiple fairness perspectives on the same dataset by changing protected attributes, outcome variables, or projection strategies.

The notebook is optimized for accessibility and reusability. It supports both causal inference students, especially those using the *Causal Artificial Intelligence* textbook [5], and applied researchers outside the causal inference community who may have rich domain knowledge and proprietary data, but limited technical infrastructure. Each section is designed to be intuitive, with clear examples, built-in error handling, and alignment with concepts from the Pearl Causal Hierarchy (PCH). A full analysis of the COMPAS [6] dataset is included to demonstrate typical use cases. The notebook, along with documentation and reproducible examples, is available at: `https://github.com/hhaller0826/ci2-fairness-ncm`.

1 Introduction

Recent advances in Causal Inference and Causal AI have made it possible to rigorously analyze complex relationships between variables and generate counterfactual explanations for social phenomena in ways that were previously out of reach. The introduction of Neural Causal Models (NCMs) for counterfactual identification and estimation [2] allows researchers to answer counterfactual questions armed with only observational data. The recently introduced framework for causal fairness analysis [3] makes it possible to find mathematical evidence of discrimination using counterfactual data. These tools offer immense promise for understanding the structural forces that shape people's lives, especially those of underrepresented or marginalized groups. Yet, despite their power, these tools remain largely inaccessible to the very researchers most intimately engaged with these questions: scholars in public health, sociology, and related fields who are collecting rich, real-world data but may not have the time, training, or technical infrastructure to implement causal models from scratch.

This notebook introduces a user-friendly framework for performing causal fairness analysis using neural causal models. It allows users to define and train a Neural Causal Model (NCM), project that model onto a Standard Fairness Model (SFM), and conduct fairness analysis on the projection. By decoupling model training from fairness analysis, this tool enables multiple fairness evaluations on the same data—with different protected attributes, outcome variables, or fairness tasks—without retraining the underlying model. This makes it highly reusable and efficient for both educational and research settings.

This notebook is designed with two primary audiences in mind:

Preprint. Under review.

- **Students of causal inference**, particularly those following the *Causal Artificial Intelligence* textbook [5]; and
- **Researchers outside the field of causal inference** who may want to apply fairness tools to their own data without needing to build a new pipeline from scratch.

While causal fairness analysis is a growing area of research, the number of researchers specializing in causal inference remains relatively small. At the same time, groundbreaking fairness work often requires access to context-rich, domain-specific datasets that are typically collected by social scientists or applied researchers. To bridge this gap, this notebook makes it simple for collaborators to plug in data, define sensitive attributes, and run fairness diagnostics. By lowering the technical barrier to entry, this work aims to support interdisciplinary collaboration and make causal fairness analysis a practical, sustainable part of long-term research into systemic inequality.

The workflow has been optimized for simplicity and readability. Creating a causal diagram only requires 2–4 lines of code, depending on whether they're using a built-in graph or creating a custom one. Processing data and graph node assignments only requires 5 lines of code, plus any preprocessing the user decides to do. Initializing and training the model only requires 2-3 lines of code, and SFM projection likewise only requires 2 lines of code. All together, the user can create a causal diagram, initialize a structural causal model, train neural networks, and project the resulting neural causal model onto the standard fairness model in roughly 10 lines of code. Running analyses is equally lightweight, with each type of query only requiring a single line of code. Additionally, the notebook includes clear guardrails to prevent common errors and provides digestible examples, including a worked analysis of the COMPAS [6] dataset, to guide users through each step.

The notebook is organized as follows:

1. **Create a Causal Graph**. Here users are walked through the process of instantiating a causal diagram to represent the relationships in their data.
2. **Train the model**. Here users are walked through loading and processing their data files, assigning data features to the nodes of the graph they created in step (1), instantiating a Neural Causal Model, and training that model.
3. **Extract Causal Insights from the Model**. This allows users to run queries from all three levels of Pearl Causal Hierarchy (PCH) [5].
4. **Project onto the Standard Fairness Model**. Here users will project the graph (and resulting model) that they created in steps (1) and (2) onto the SFM.
5. **Run Fairness Tasks** This will walk users through performing causal fairness tasks such as bias detection, bias quantification, and fair prediction.

By simplifying the technical overhead and emphasizing modularity, this tool empowers users to focus on the core analytical questions, making causal fairness analysis more accessible, iterative, and collaborative across disciplines.

2 Notebook Section 1: Create a Causal Graph

The notebook begins with graph specification to get users to conceptualize their analysis in terms of interactions between model variables, rather than the relationships between their data columns. The introduction information emphasizes that the analysis will be run based on the variables, and that if multiple data features are assigned to a single node/variable, the notebook will not be able to distinguish between those features when doing causal analysis.

Users can define causal graphs in one of two ways:

1. **Predefined Graph Choices**. A set of predefined graphs is available, including *simple*, *bow*, *frontdoor*, *backdoor*, *napkin*, and *SFM*. These were chosen because they are commonly referenced in the Causal AI textbook [5], so anyone using this notebook as supplemental material for the textbook will find them familiar. Each graph is easily digestible, and clearly demonstrates the graph-building concepts and requirements described in the instructions, progressing in complexity to serve as instructive examples for users interested in making custom graphs.

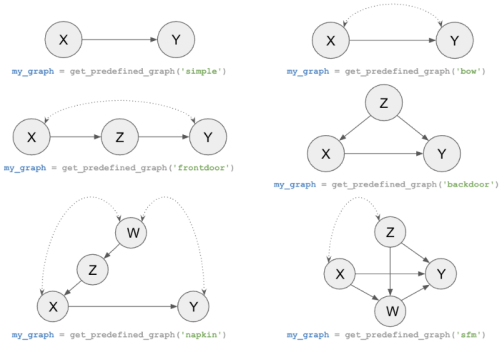


Figure 1: Default graph options

2. The user can create a custom graph by defining a set of nodes, bidirected edges, and directed edges. The example provided in the accompanying markdown cell is much simpler than the one in the actual code cell. The markdown cell prioritizes clarity and approachability, whereas the code cell contains a graph tailored for the COMPAS dataset, and is designed to accommodate multiple SFM projections.

At the end of the graph-building section, a summary of structural requirements is provided. However, most of these constraints are semantic and rely on domain-specific knowledge, and thus cannot be fully validated programmatically.

Two key exceptions are enforced at runtime:

- If an edge references a node not defined in the node list, a `KeyError` is raised, explicitly identifying the offending node.
- If the graph contains cycles, a `ValueError` is raised with the message: `ValueError: The graph cannot have cycles.`

Duplicate edges are allowed without issue (e.g. specifying "`directed_edges=[(X,Y),(X,Y)]`" will not result in an error and will simply register the edge $X \to Y$).

Once the graph is defined, users are able to plot it so that they can visualize the structure they've created and ensure it looks how they expected. The plotting utility arranges nodes in a circular layout based on their topological order, providing a clean and interpretable display across a wide range of graph types. To minimize complexity and preserve usability for novice users, manual specification of node positions is not supported. This default layout is sufficient for most use cases, particularly those where users are still building an intuition for causal graph structures.

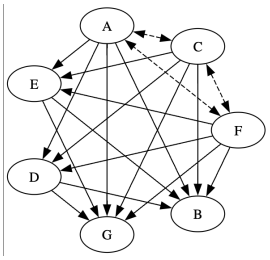


Figure 2: "Custom" graph created in the code walk-through

3 Notebook Section 2: Train the Model

3.1 Process Data and Assignments

Before training the neural causal model, users must process their dataset and assign features to variables defined in the causal graph. This section of the notebook walks users through these steps in

a clear and consistent manner. Each step in this process is first explained in a markdown cell and then mirrored exactly in the code cell below to ensure clarity and reproducibility. The steps are as follows.

1. Initialize a `DataFrame` object and do any special preprocessing that's needed.
2. Specify which data features have categorical or discrete values.
3. Assign data features to nodes in the graph that was created in Section 1. Assignment will be a dictionary of the form

```
assignments = {
    'node_1': ['data_feature','data_feature',...],
    'node_2': ['data_feature','data_feature',...],
    ...
}
```

3.1.1 Checking Assignments

A series of guardrails and validation checks are in place to help users correctly assign features and avoid common mistakes. It will raise a `ValueError` if any of the following are violated:

1. All keys in the assignment dictionary must correspond to a node on the graph.
2. All graph nodes must have at lease one data feature assignment, empty lists are not allowed.
3. All data features being assigned must exist in the `DataFrame` object that was just loaded.

If there are features in the `DataFrame` object that were not assigned to any node, then no errors are thrown but the following warning is shown in case features were unintentionally omitted:

```
UserWarning: The following features were not assigned to any variable: {...}
    It is okay to exclude features from the model but they will not be
    used in the causal analysis.
```

3.1.2 Processing Data

Once the assignments have been checked, they and the `DataFrame` object are used for data processing. Processing the data involves splitting it into train and test sets, converting all of the categorical values into integers, and scaling all of the data down into a 0-1 range. This scaling helps to constrain the model's outputs to a meaningful range across variable types. Without such constraints, the model occasionally produces predictions well outside the expected bounds, especially for categorical values that inherently fall within a finite set.

This processing also includes creating a map to scale the data back to its original range. Although translation back from these representations does not include mapping to original category names. This design choice improves robustness in downstream user tasks (such as the Level 1–3 queries in Section 3), as numeric types (e.g., int or float) are less error-prone than string types, which are more susceptible to formatting errors, typos, or incorrect data types.

The utility `print_df()` allows users to preview the processed dataset and optionally compare it to the original input values, which is especially valuable for categorical features which will continue to hold numeric values. Options with and without comparison are shown below.

	race	score_text	age	juv_fel_count	juv_misd_count	juv_other_count	priors_count	sex	two_year_recid
308	1.0	0.5	0.564103	0.0	0.000000	0.0	0.000000	1.0	0.0
381	0.0	0.5	0.128205	0.0	0.000000	0.0	0.026316	1.0	0.0
3238	0.0	0.5	0.500000	0.0	0.000000	0.0	0.210526	1.0	1.0
2312	0.0	0.0	0.012821	0.0	0.076923	0.0	0.026316	1.0	1.0
251	0.0	0.5	0.243590	0.0	0.000000	0.0	0.000000	0.0	0.0

race_orig	race	score_text_orig	score_text	age_orig	age	juv_fel_count_orig	juv_fel_count	juv_misd_count_orig	juv_misd_count	juv_other_count_orig	juv_other_count	priors_co
1	1.0	Low	0.5	40	0.282051	0	0.0	0	0.0	0	0.000000	
1	1.0	Low	0.5	32	0.179487	0	0.0	0	0.0	1	0.058824	
0	0.0	High	0.0	41	0.294872	0	0.0	0	0.0	0	0.000000	
0	0.0	High	0.0	32	0.179487	0	0.0	0	0.0	0	0.000000	
0	0.0	Medium	1.0	36	0.230769	0	0.0	0	0.0	0	0.000000	

3.2 Initialize and Train the NCM

The user goes on to create an NCM based on the graph that they created in Section 1, which is then trained using the data that was processed earlier in this section.

Definition 3.1 ($\mathscr{G}$-Constrained NCM) *Let $\mathscr{G}$ be the causal Diagram induced by SCM $\mathscr{M}$*, and $\mathbf{V} := \mathscr{G}(\mathbf{V})$ be the endogenous variables of $\mathscr{M}$*. Construct NCM $\hat{M}$ as follows.*

1. *Choose $\hat{\mathbf{U}}$ such that for each subset $\mathbf{C} \subseteq \mathbf{V}$, there is an associated exogenous variable $\hat{U}_{\mathbf{C}} \in \hat{\mathbf{U}}$ if and only if every pair $V_i, V_j \in \mathbf{C}$ is connected via a path of bi-directed arrows in $\mathscr{G}$ and is maximal (i.e. there will not be no exogenous variable $\hat{U}_{\mathbf{C}'} \in \hat{\mathbf{U}}$ for which $\mathbf{C} \subseteq \mathbf{C}'$).*
2. *For each variable $V_i \in \mathbf{V}$, choose $\mathbf{Pa}_{V_i} \subseteq \mathbf{V}$ such that for every $V_j \in \mathbf{V}$, $V_j \in \mathbf{Pa}_{V_i}$ if and only if there is a directed edge from V_j to V_i in $\mathscr{G}$.*

Any NCM in this family is said to be $\mathscr{G}$-constrained. For simplicity, a $\mathscr{G}$-constrained NCM will be shortened to $\mathscr{G}$-NCM. [5; Definition 13.1.10-13.1.11]

The user is able to use the `hyperparameters` dictionary to specify which probability distribution they'd like in their model, the type of neural network to be used for each $\hat{f}$, the device, number of epochs to train for, learning rate, and optimizer. A $\mathscr{G}$-NCM is then instantiated.

By the soundness and completeness of Neural Counterfactual Identification [2], an NCM $\hat{M}$ can answer causal questions about real world data so long as the probability distribution produced by $\hat{M}$ is sufficiently close to the probability distribution of the data, and they can both be described by the same causal diagram.

Algorithm 1 NCM training pseudo-code

```
procedure TRAIN-NCM(data=D,𝒢)
    M̂ ← NCM(V,𝒢)
    Initialize parameters θ
    for each epoch do
        loss ← 0
        for each batch of real datapoints in D do
            P(V) ← probability distribution of this batch
            P̂(V) ← M̂(θ).sample(n)                ▷ where n is the size of this batch
            loss ← loss + divergence between P(V) and P̂(V)
        end for
        Update θ based on loss
    end for
end procedure
```

The algorithm above is a stripped-down version of Algorithm 3 in [2]. Unlike [2], this notebook assumes that the inputted data will be Level-1, observational data, and does not have added support for training on interventional datasets. It also does not specify a query. Query specification in [2; Algorithm 3] exists only to track how a model's evaluation of the query changes from epoch-to-epoch. The goal of this notebook is to train a model that can then be used to compute many queries, so specifying a query prior to training is unnecessary and may motivate users to come up with new queries and re-train, which would be counterproductive.

To evaluate the divergence for loss calculations during training there needs to be a divergence function that can work with samples directly. This ruled out many popular divergence metrics like Kullback-Leibler, which measures how a probability distribution differs from a true probability. Popular sample-based divergence metrics include maximum mean discrepancy (MMD), Wasserstein distance, energy distance, k-nearest neighbors, and classifier-based divergences. Of those, the k-nearest neighbors algorithm does not handle high dimensions well, and Wasserstein is computationally expensive. I implemented an energy based divergence method, MMD, and Jensen-Shannon divergence (which

is a type of classifier-based divergence). I used MMD to train because it seemed to be the most discriminatory of the three.

4 Notebook Section 3: Extract Causal Insights

This part of the notebook has three subsections, each corresponding to a layer of the PCH.

4.1 Observational Understanding

The notebook does more or less assume that users have a basic understanding of Level 1 queries, namely probability $P(Y)$ and conditional probability $P(Y|X)$ notation, however it does not assume prior knowledge of more specific quantities like "total variation". Details of the implemented Level 1 queries are described below.

Here users are able to compute a probability or conditional probability with the function called "`probability(model, variable, value, evidence)`" where `model` is the trained NCM, `variable` specifies the name of the outcome node, `value` is the outcome we are interested in, and `evidence` is a dictionary object specifying the conditionals – each key is the name of a node which maps to the value it is meant to assume.

- "`probability(model, 'Y', 1)`" will output P(Y=1).
- "`probability(model, 'Y', 1, {'X':0,'Z':2})`" will output P(Y=1 | X=0, Z=2).

Total variation measures how much the probability distribution of a given variable is impacted by the observed value of some other attribute, and is calculated by the equation below [5]:

$$TV_{x_0,x_1}(y) = P(y|x_1) - P(y|x_0)$$

The function "`total_variation(model, variable, value, attr, aval1, aval0)`" computes the total variation of the given variable relative to the given attribute, and returns $TV_{aval0,aval1}(variable = value)$. "`aval1`" is an optional value, and will default to "`not aval0`" if left blank.

- "`total_variation(model, 'Y', 1, 'X' 0, 1)`" will output $TV_{X=0,X=1}(Y = 1) = P(Y = 1|X = 1) - P(Y = 1|X = 0)$.
- "`total_variation(model, 'Y', 1, 'X' 0, 1)`" will output $TV_{X\neq 1,X=1}(Y = 1) = P(Y = 1|X = 1) - P(Y = 1|X \neq 1)$

The practice of allowing users to leave a certain conditional value blank, allowing it to default to "`not {some_other_value}`" is used throughout this notebook. This way, if an attribute is non-binary, the user is still able to evaluate the impact of that attribute taking on a specific value. This is possible because to evaluate a Layer 1 probability $P(Y = 1)$, n points are sampled from the trained model, and the output is the frequency with which $Y = 1$ in the sample. To evaluate a conditional probability $P(Y = 1|X = x)$, n points are sampled, then any samples for which $X \neq x$ are filtered out, and again output the frequency with which $Y = 1$ in the sample. "`not {some_other_value}`" can be evaluated by instead filtering out the values for which the attribute does in fact equal "`{some_other_value}`".

4.2 Interventional Metrics

Interventional queries are introduced using an example where visiting a doctor gives patients access to advice and medication, which in turn affects how quickly they are healed. The causal diagram for this example is the same as the one used in the sprinkler example that accompanies the explanation of interventional queries in [5], providing continuity for readers familiar with the textbook. However, the example is framed in a more applied and behaviorally intuitive context to enhance accessibility for users primarily interested in research applications. Grounding the concept of an intervention in familiar human experiences makes it more intuitive for users who are unfamiliar with or recently introduced to the PCH. Additionally, it sets the reader up for more medical examples which are used in Layer 3.

To evaluate an interventional query, something like $P(Y = 1|do(X = x), Z = z)$, you'd follow the same steps as above, except when sampling from the model, you'd force $X = x$ instead of calculating $X = f_X$, and then filter out values for which $Z = z$. Because of this, in the Layer 2 queries, users are still able to put `neq` statements in the conditions, but the interventions $do(X = x)$ must be explicit.

Here users are given the option to compute interventional probabilities using the same `probability` function as before, but with an added `intervention` parameter that specifies a dictionary of the interventions similar to `evidence`.

- `probability(model,'Y',1,{'X':0},{'Z':1})` will return $P(Y = 1|X = 0, do(Z = 1))$
- `probability(model,'Y',1,intervention={'Z':1})` will return $P(Y = 1|do(Z = 1))$

Users are then introduced to the concept of *Total Effect*, and given the option to compute it with the function `"total_effect(model, variable, value, attr, aval1, aval0, evidence)"` which returns $TE_{aval0,aval1}(variable = value|evidence)$.

- `total_effect(model, 'Y', 1, 'X', 1, 0)` will return $TE_{0,1}(Y = 1) = P(Y = 1|do(X = 1)) - P(Y = 1|do(X = 0))$.
- `total_effect(model, 'Y', 1, 'X', 1, 0, evidence={Z:z})` will return z-$TE_{0,1}(Y = 1) = P(Y = 1|do(X = 1), Z = z) - P(Y = 1|do(X = 0), Z = z)$.

4.3 What-if? Quantifying the Imaginary

To adequately explain the concept of a counterfactual this section uses another medical example, introduces some new notation, and mirrors textbook [5] and *Book of Why* [4] descriptions of the concept. The medical example ties this section back to the interventional query portion, it mirrors examples in the textbook [5], and the language around the most common counterfactual queries naturally lends itself to a medical setting. Notably, the first counterfactual query that gets introduced is the ETT (*Effect of the Treatment on the Treated*) and the concept of "treatment" and entities which are "treated" make the most linguistic sense in a medical context.

Before diving into the specific queries users are able to compute, I did have to explain the significance of $P(Y_X)$ notation. Details of the implemented Layer 3 queries are described below.

Users can compute the effect of the treatment on the treated by calling `"ett(outcome_var, outcome_val, treatment_var, treatment_vals)"` where `treatment_vals={'actual':_, 'whatif':_}`. The `"treatment_vals"` dictionary must have a `'whatif'` key but the `'actual'` can be unspecified. This reflects the `neq` requirements stated in the interventional metrics section.

- `ett(model, 'Y',1,'X',{'actual':0, 'whatif':1})` will return $P(Y_{X=1} = 1|X = 0)$.
- `ett(model, 'Y',1,'X',{'whatif':1})` will return $P(Y_{X=1} = 1|X \neq 1)$.

The *probability of necessity/sufficiency* is described similarly, and can be computed with `"pnps(model, outcome_var, outcome_vals, treatment_var, treatment_vals)"` where `"outcome_vals"` and `"treatment_vals"` are both dictionaries of the form `{'actual':_, 'whatif':_}`. Once again, both `'whatif'` keys are required, but the `'actual'` values can be unspecified.

- The function call

  ```
  pnps(model, 'Y', outcome_vals={'actual':0, 'whatif':1},
                          'X', treatment_vals={'actual':0, 'whatif':1})
  ```

  will return the probability that $Y = 1$ if we had intervened to make $X = 1$, given that Y and X were actually both 0.

Sampling for counterfactual queries is more complicated than sampling for level 1 or 2 queries. To calculate something like $P(Y_{X=x_0}|X = x_1, Z = z)$ you would first sample the exogenous variables, feed them into the model to get a full probability distribution $P(V)$, filter out values for which $X \neq x_1$ or $Z \neq z$, and then re-sample the model by feeding it the filtered exogenous variables and forcing $X = x$. Once that is done, you can look for the frequency with which $Y = 1$ in the sample. This basic counterfactual sampling procedure is detailed in the algorithm below. This is based on algorithms from [2].

Algorithm 2 Basic Counterfactual Sampling

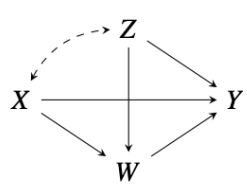
```
procedure SAMPLE-CTF-BASIC(scm, term=CTFTerm, conditions=None, u=None,
n=10000)
    U ← Conditioned-U(scm, u, conditions, n)
    sample←scm.sample(u=U,do=term.do-values)
    return sample
end procedure

procedure CONDITIONED-U(scm, u=None, conditions=None, n=10000)
    if u is None then
        U ← scm.pu.sample(n)
    else
        U ← u
        n ← len(u.samples)
    end if
    if conditions is None then return U
    end if
    sample ← scm.sample(u=U)
    indices-to-keep ← set()
    for c in conditions do
        temp-indices ← indices where sample[c]==conditions[c]
        indices-to-keep ← indices-to-keep ∩ temp-indices
    end for
    return U[indices-to-keep]
end procedure
```

5 Notebook Section 4: SFM Projection

One of the core strengths of this notebook lies in its reusability: users can project a fully trained neural causal model (NCM) onto multiple variations of the Standard Fairness Model (SFM) without retraining the model. This makes it possible to analyze fairness under different configurations of sensitive attributes, mediators, and outcomes—all derived from the same base model.

Definition 5.1 (Structural Fairness Model (SFM)) *The Structural Fairness Model refers to a class of SCMs $\Omega(\mathcal{G}_{SFM})$, where $\mathcal{G}_{SFM}$ is a causal diagram over endogenous variables $\{X, Z, W, Y\}$ given by*



where the nodes represent:

X: *the protected attribute,*

Z: *the set of confounding variables, which are not causally influenced by the attribute X,*

W: *the set of mediator variables that are possibly causally influenced by the attribute,*

Y: *the outcome variable.*

For any model $\mathscr{M} \in \Omega(\mathscr{G}_{SFM})$, Z and W may be multi-dimensional or empty, but X and Y must be one-dimensional and explicitly defined [5; Definition 6.2.6]. For simplicity, a $\mathscr{G}_{SFM}$-NCM will be shortened to SFM-NCM.

The absence of arrows in an SCM are more important then the presence of arrows, so the structure of the SFM is explained to users in terms of which paths need to be avoided (rather than which ones are typically present). Namely,

- There can be no paths from Y to X, Z, or W.
- There can be no paths from W to X or Z.
- There can be no *directed* paths from X to Z or Z to X, even though there may be bidirected paths.

Definition 5.2 (SFM Projection) *The projection $\Pi_{SFM}(\mathscr{G})$ of a causal diagram $\mathscr{G}$ is the mapping of the endogenous variables $\mathscr{G}(\mathbf{V})$ onto $\mathscr{G}_{SFM}$. The mapping can be written as*

$$\Pi_{SFM}(\mathscr{G}) =< X = \{V_x\}, Z = \{V_{z_1}, V_{z_2}...\}, W = \{V_{w_1}, V_{w_2}...\}, Y = \{V_y\} >$$

for $\{V_i\} \in \mathscr{G}(\mathbf{V})$. To get the variables from the initial graph, we can define the "reverse" mapping

- $\Pi_{SFM}(\mathscr{G})[X] \rightarrow \{V_x\}$
- $\Pi_{SFM}(\mathscr{G})[Z] \rightarrow \{V_{z_1}, V_{z_2}, ...\}$
- $\Pi_{SFM}(\mathscr{G})[W] \rightarrow \{V_{w_1}, V_{w_2}, ...\}$
- $\Pi_{SFM}(\mathscr{G})[Y] \rightarrow \{V_y\}$

Note that in code, this is effectively describing the definition of a dictionary $\Pi_{SFM}(\mathscr{G})$ whose keys are the SFM variables $\{X, Z, W, Y\}$ and whose values are the elements of $\mathscr{G}$ being projected onto each variable. [5; Definition 6.2.6]

There are multiple ways that a given causal diagram can be projected onto the SFM, and this may be confusing for users, so the markdown instructions have an example of a causal diagram that is projected onto the SFM in four different ways.

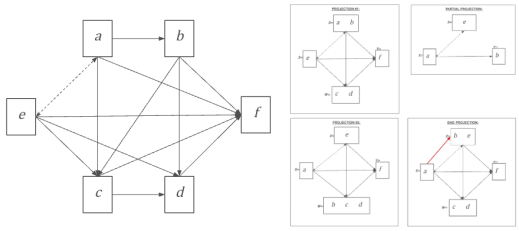


Figure 3: Projection examples shown to users in notebook section 4

The diagram was chosen to be similar to the SFM in Causal AI book Chapter 6, but simpler and specifically designed to have clear examples of multiple different acceptable projections, as well as an unacceptable projection that looks similar to the acceptable ones. The unacceptable projection looks very similar to the acceptable ones, and is likely to be mistaken for an acceptable projection by someone new to the field, but should be simple to understand once it is pointed out.

5.1 User Requirements

This stage is particularly prone to user error, so the notebook includes a comprehensive set of safeguards to guide users through the process of projecting their graph onto the SFM. The projection is defined as a dictionary `projection = {'X':_, 'Z': [_], 'W': [_], 'Y':_}`. Errors and warnings are raised as follows.

5.1.1 Checking Keys and Values

A value error will be raised if X or Y is not specified, like so:

```
AssertionError: Must specify a protected attribute X.
AssertionError: Must specify an outcome Y.
```

If W or Z is unspecified, it will let the user continue with the warning

```
UserWarning: No confounders assigned.
UserWarning: No mediators assigned.
```

Having an additional key will throw a warning, not an error. So long as the rest of the projection meets the requirements for the SFM it's not really an issue, so long as the user is made aware. So if the user adds `{'extra_key':'some_node'}` to their projection, but their X, Z, W, and Y assignments are solid, the following warning will be shown.

```
UserWarning: {'extra_key'} will not be used as variables in the SFM.
    The following will remain unassigned: some_node
```

There is however a `ValueError` if they try to project a value that was not in the original graph onto one of the actual variables of the SFM.

5.1.2 Checking Paths

If the user is projecting a subgraph of their trained NCM onto the SFM, the notebook requires that they project the complete graph. So, for example, in Figure 3, if they projected `a` and `f` onto the SFM, they would also have to project `b`. This is in part to avoid weird edge cases, but also for user comprehension. If the user creates a projection $\Pi_{SFM}(\mathscr{G})$ such that $\Pi_{SFM}(\mathscr{G})[W] = \{W_1, W_2\}$ and there is some node V such that there is a path $W_1 \to V \to W_2$ in $\mathscr{G}$, then the causal properties and relationships associated with V will be projected onto W in the resulting SFM-NCM. The same would have to be true if V existed along some path $X \to V \to W_i$ or $W_i \to V \to Y$. This also applies to variables along paths within $\Pi_{SFM}(\mathscr{G})[Z]$. Either way, relationships that exist along paths within the SFM will be reflected in the resulting model, so it's best that the user explicitly adds all if these variables to their projection so that they can see and understand what's happening. It will also prevent users from trying to be "sneaky": if there is a graph with path $Z_1 \to V \to Z_2$ and edge $X \to V$, the user may think they can get around SFM path restrictions by simply refusing to project V onto the SFM, and fail to realize that there is still a causal relationship $X \to Z_2$. Failure to do so will result in a warning similar to the one below.

```
ValueError: [Path {'X', 'Z1'}->V->{'Z2'}]: Please explicitly assign E
to one of the SFM variables.
```

It then checks that there are no bidirected paths between Y and any of the other variables, or between W and X, Z. It also checks the ancestors of these variables that were left out of the graph. If there is some variable V which is an ancestor of both $\Pi_{SFM}(\mathscr{G})[X]$ and $\Pi_{SFM}(\mathscr{G})[W]$ but is not a part of the projection, then a bidirected edge has effectively been created between X and W. In these cases, the user will see something like one of the following.

```
ValueError: Cannot have bidirected arrow path between X or Z and Y.

ValueError: Unassigned node(s) {V} point to Y and other variables in your
projection. This is the same as having a bidirected arrow path.
```

The latter case is not explicitly stated in the instructions because it may be confusing for users who cannot conceptualize it, but if they make the mistake of introducing such a confounder the error should be clear enough to explain the issue.

Lastly, we check the basic SFM arrow requirements which, if violated, will raise the following error(s).

```
ValueError: Cannot have path from Y to X, Z, or W in the Standard Fairness Model.
```

```
ValueError: Cannot have path from W to X or Z in the Standard Fairness Model.
ValueError: Cannot have path from Z to X in the Standard Fairness Model.
ValueError: Cannot have path from X to Z in the Standard Fairness Model.
```

5.2 Constructing the Projection

Users can create a multitude of SFM-NCMs by training a single model $\hat{M} \in \Omega(\mathscr{G})$ and constructing each SFM-NCM by creating a mapping $\Pi_{SFM}(\mathscr{G})$ and applying it to $\hat{M}$. One of the core strengths of this notebook lies in its reusability, so each resulting SFM-NCM should re-use as much of the $\hat{M}$'s structure as possible.

Given a $\mathscr{G}$-NCM $\hat{M} =< \hat{\mathbf{U}}, \mathbf{V}, \hat{\mathscr{F}}, \hat{P}(\hat{\mathbf{U}}) >$ where $\hat{\mathscr{F}} = \{\hat{f}_V : V \in \mathbf{V}\}$ and a projection $\Pi_{SFM}(\mathscr{G})$, the resulting SFM-NCM will be as follows.

$$
\begin{aligned}
\mathbf{V}_{SFM} &= \{X, Z, W, Y\} \\
\hat{\mathbf{U}}_{SFM} &= \hat{\mathbf{U}} \\
\hat{\mathscr{F}}_{SFM} &= \begin{cases} \hat{f}_X(\hat{\mathbf{u}}) \leftarrow \hat{f}_x(\mathbf{pa}_x, \hat{\mathbf{u}}); x = \Pi_{SFM}(\mathscr{G})[X] \\ \hat{f}_Z(\hat{\mathbf{u}}) \leftarrow \{\hat{f}_z(\mathbf{pa}_z, \hat{\mathbf{u}})\}_{z \in \Pi_{SFM}(\mathscr{G})[Z]} \\ \hat{f}_W(x, z, \hat{\mathbf{u}}) \leftarrow \{\hat{f}_w(\mathbf{pa}_w, \hat{\mathbf{u}})\}_{w \in \Pi_{SFM}(\mathscr{G})[W]} \\ \hat{f}_Y(x, z, w, \hat{\mathbf{u}}) \leftarrow \hat{f}_y(\mathbf{pa}_{\Pi_{SFM}(y}, \hat{\mathbf{u}}); y = \Pi_{SFM}(\mathscr{G})[Y] \end{cases} \\
\hat{P}(\hat{\mathbf{U}}_{SFM}) &= \hat{P}(\hat{\mathbf{U}})
\end{aligned}
\tag{1}
$$

To manage the reuse of neural networks $\hat{\mathscr{F}}$ in a modular and interpretable way, this notebook uses two structural patterns:

- **Vertical Stack** (for $\hat{f}_X$ and $\hat{f}_Z$): a *vertical stack* is a composition of functions alignes along the ancestral hierarchy, computing variables in topological order from exogenous sources to each SFM node. This ensures that each node has access to all of its required inputs without recomputing shared ancestors multiple times.
- **Horizontal Stack** (for $\hat{f}_W$ and $\hat{f}_Y$): a *horizontal stack* manages parallel composition across multiple target nodes, each of which may have different parents. If those parents include variables that were not projected to the SFM, a single vertical stack is created to handle all required upstream computations. From the perspective of the horizontal stack, this vertical component is treated as a single element which is in the same "horizontal" lineup as the SFM parents X, Z, and maybe W.

[**Example 5.2.1**]: Given causal diagram $\mathscr{G}$ below,

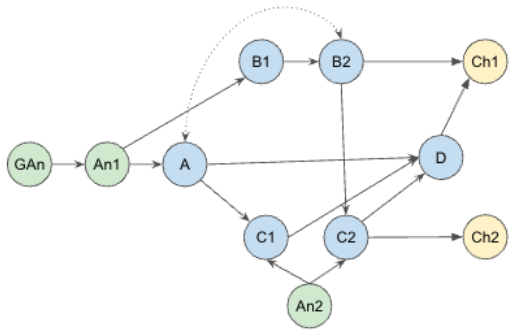
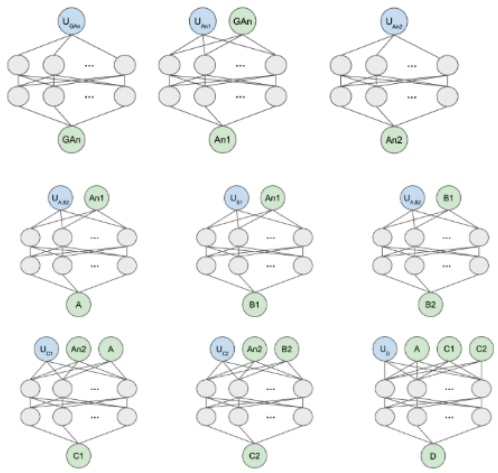


Figure 4: Causal diagram for $\hat{M}$

Let $\hat{M}$ be a $\mathscr{G}$-NCM. Then $\hat{M}$ has endogenous variables $\mathbf{V}$ = `{'GAn','An1','An2','A','B1','B2','C1','C2','D','Ch1','Ch2'}` and exogenous

variables $\hat{\mathbf{U}} = \{U_{\texttt{GAn}}, U_{\texttt{An1}}, U_{\texttt{An2}}, U_{\texttt{A,B2}}, U_{\texttt{B1}}, U_{\texttt{C1}}, U_{\texttt{C2}}, U_{\texttt{D}}, U_{\texttt{Ch1}}, U_{\texttt{Ch2}}\}$. Lets assume $\hat{M}$ is already trained, and that each function is a multi-layer perceptron with two hidden layers.

Given the projection $\Pi_{SFM}(\mathscr{G}) =< X = \{\texttt{A}\}, Z = \{\texttt{B1,B2}\}, W = \{\texttt{C1,C2}\}, Y = \{\texttt{D}\} >$, the neural networks of $\hat{M}$ pertaining to the projected nodes and their ancestors are shown below.



To create $\hat{f}_X$, we need a structure that can take in $\hat{\mathbf{u}}$ and output A using the networks displayed above. $\hat{f}_Z$ is created the same way, except it must output Z as a concatenation of B1 and B2.

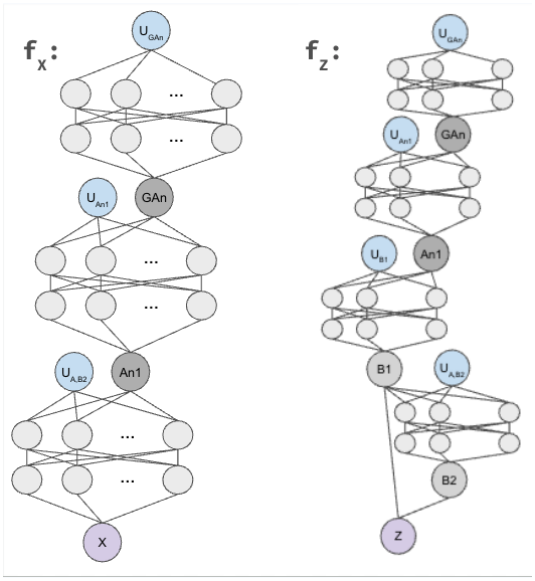


Figure 5: Vertical stacks f_X and f_Z

$\hat{f}_W$ must take in $\hat{\mathbf{u}}, X, Z$ and output W which is a concatenation of C1 and C2. To do this, it needs to convert (X, Z) into $(\texttt{A}, \texttt{B1}, \texttt{B2})$ and use $\hat{\mathbf{u}}$ to calculate An2. Similarly, $\hat{f}_Y$ must take in $\hat{\mathbf{u}}, X, Z, W$ and output D, which requires converting (X, Z, W) into $(\texttt{A}, \texttt{B1}, \texttt{B2}, \texttt{C1}, \texttt{C2})$ and outputting D. Because all of D's parents were projected onto the SFM, f_Y does not need to use $\hat{\mathbf{u}}$ beyond extracting the value $u_{\texttt{D}}$.

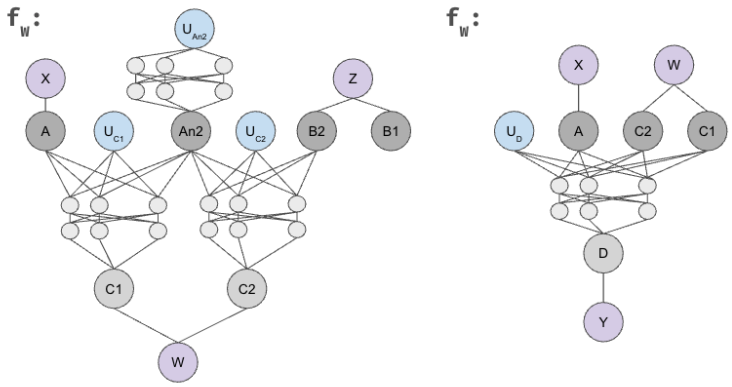


Figure 6: Horizontal stacks f_W and f_Y

We only need to worry about variables that were projected onto the SFM and their ancestors, because the values and functions associated with `Ch1` and `Ch2` have no bearing on the values within the SFM. It is necessary to handle the ancestors because reusing the exogenous probability distribution means that we cannot directly plug $\hat{\mathbf{U}}$ into an SFM function like $\hat{f}_X$ if $\hat{f}_{\Pi_{SFM}(\mathscr{G})[X]}$ is expecting additional parent values. This modular reuse imposes no additional training cost and incurs only minimal overhead—primarily due to potential recomputation of shared ancestors, which is rare and negligible in practice.

5.3 Understanding the results

Once the user has successfully projected their graph to the SFM, they can choose to print out the result of their projection, which will relate the *data* features to SFM attributes like so:

```
Protected Attribute: ['race']
Confounders:         ['age', 'sex']
Mediators:           ['priors_count', 'juv_fel_count', 'juv_misd_count']
Outcome:             ['two_year_recid']
```

This should help them understand how the SFM relationships pertain to the real-world information in their dataset.

6 Notebook Section 5: Causal Fairness Analysis

The examples in this section primarily relate to human demographics and hiring decisions because that is one of the most common and easy to understand applications of fairness tools, it mirrors the examples used in textbook section 6.4 [5], it lends itself to the language of disparate treatment and disparate impact, and it makes concepts like "business necessity" more intuitive.

6.1 Bias Detection and Quantification

There are two applications of bias detection and quantification. The first is TV-Decomposition, where one can compute direct, indirect, and spurious effects based on probabilities. $NDE = P(Y_{x_1 W_{x_0}}) - P(Y_{x_0})$, $x\text{-}DE_{x_0,x_1}(y|x) = P(y_{x_1 W_{x_0}}|x) - P(y_{x_0}|x)$, etc. These are valuable for theory-based applications, and to get a general quantification of these effects in an applied setting.

However, oftentimes when we are doing bias detection and quantification on real data, it is to run the Fairness Cookbook [3], which aims to accept or reject hypotheses about whether or not each of these effects is zero. In this case, one would get the 95% confidence interval of the expected value of Y, rather than the probability that Y takes on a particular value. If zero is not inside of the resulting confidence interval of the direct, indirect, or spurious effect, then the associated hypothesis will be rejected, indicating that there is evidence of discrimination.

Both techniques start with counterfactual sampling with nested counterfactual $Y_{x,W_{x'}}$.

To compute nested counterfactuals, one would follow the basic counterfactual sampling procedure from Algorithm 2 to filter out samples where $Z \neq z$ and get a list of samples for W_{x_0}. Then you'd go back to the sample with $Z \neq z$ filtered out and get another sample for the model, except instead of calculating $X = f_X$, you'd force $X = x_1$, and instead of calculating $W = f_W$, you'd force W to take on the values from the previously sampled list. In this way, you force $X = x_1$ and $W = W_{x_0}$, even if W_{x_0} varies depending on the provided U value. Algorithm below also based on [2].

Algorithm 3 Counterfactual Sampling

```
procedure SAMPLE-CTF(scm, term=CTFTerm, conditions=None, u=None, n=10000)
    U ← Conditioned-U(scm, u, conditions, n)
    expanded-dos ← dict()
    for k in term.do-values do
        if k is nested then          ▷ Calculate nested counterfactual, update related variable
            ctf-sample = Sample-CTF(scm, k.term, U)
            expanded-dos[k.term]←ctf-sample[k.term])
        else                                  ▷ Force related variable to take a given value
            expanded-dos[k.term]←term.do-values[k]
        end if
    end for
    sample←scm.sample(u=U,do=expanded-dos)
    return sample
end procedure
```

Once sampled, depending on the application, we will either get the probabilistic TV-Decomposition, or the 95% confidence interval TV-Decomposition.

The probabilistic one can be calculated the same as other probabilities, where $P(y_{x,W_{x'}}|x)$ can be computed by doing the required sampling, then finding the frequency with which $Y = y$ in the sample. Once these individual probability values are computed, the direct effect can be computed directly as $x\text{-}DE_{x,x'}(y) = P(y_{x,W_{x'}}|x) - P(y_{x'}|x)$, and the indirect and spurious effects can be computed similarly.

To get the 95% confidence interval, you start by doing the same counterfactual sampling. Then with the given sample, use a Bootstrapping technique [5] to get a mean value and confidence interval by re-sampling small samples from the general counterfactual sample. When this is done you'll have $\mathbb{E}[Y_{x,W_{x'}}|x] = mean(Y_{x,W_{x'}}|x) \pm ci(Y_{x,W_{x'}}|x)$, $\mathbb{E}[Y_{x'}|x] = mean(Y_{x'}|x) \pm ci(Y_{x'}|x)$, etc.

Then to calculate $x\text{-}DE$, you can subtract the mean values like normal, and get the confidence interval of the difference with $ci(DE) = \sqrt{(ci(Y_{x,W_{x'}}|x))^2 + (ci(Y_{x'}|x))^2}$. So the 95% confidence for $x\text{-}DE[Y|x] = [mean(Y_{x,W_{x'}}|x) - mean(Y_{x'}|x)] \pm \sqrt{(ci(Y_{x,W_{x'}}|x))^2 + (ci(Y_{x'}|x))^2}$.

If running the cookbook, users will see an output similar to the following.

```
 xDE^sym = -0.4482±0.0207  hypothesis "no direct effect" REJECTED
---> evidence of disperate TREATMENT.
 xIE^sym = -0.0542±0.0207  hypothesis "no indirect effect" REJECTED
 xSE_x1x0 = -0.0066±0.0162  hypothesis "no spurious effect" ACCEPTED
---> evidence of disperate IMPACT.
```

The english language should be understandable to non-causal users, since the concepts of disparate treatment and impact were already explained. The added details of specifying whether or not the hypothesis was rejected or accepted is so that book readers have a better concept of what's going on.

6.2 Fair Prediction

This section allows users to come up with a fair predictor $\hat{Y}$ of Y, which is then added to their SFM-NCM as an additional node. This additional node has function $f_{\hat{Y}} : \{X, Z, W\} \to \hat{Y}$ whose

domain does not include the exogenous variables. This is an artificially constructed predictor, so there is no need to account for unseen variables influencing it, because it is entirely based on the observed factors within the model. It is trained using the following optimized loss function [3]

$$L(y,\hat{y}) + \lambda\Big(|\text{NDE}_{x_0,x_1}(\hat{y}) - \eta_1| + |\text{NIE}_{x_1,x_0}(\hat{y}) - \eta_2| + |\text{Exp-SE}_{x_1}(\hat{y}) - \eta_3| + |\text{Exp-SE}_{x_0}(\hat{y}) - \eta_4|\Big)$$

If $DE \in$ the business necessity set BN, then $\eta_1 = 0$, otherwise $\eta_1 = NDE_{x_0,x_1}(y)$. If $IE \in BN$ then $\eta_2 = 0$, otherwise $\eta_2 = NIE_{x_1,x_0}(y)$. If $SE \in BN$ then $\eta_3 = \eta_4 = 0$, otherwise η_3 and η_4 are set to the appropriate Exp-SE value for the true outcome Y. These restrictions allow $\hat{Y}$ to try to learn classifier Y while keeping measures of discrimination close to 0.

The user is now able to get fair predictions, or run the fairness cookbook to get a TV-decomposition of their new fair classifier.

References

[1] Xia, K. & Lee, K.Z. & Bengio, Y. & Bareinboim, E. (2022) *The Causal-Neural Connection: Expressiveness, Learnability, and Inference*. https://causalai.net/r80.pdf

[2] Xia, K. & Pan, Y. & Bareinboim, E. (2023) "Neural Causal Models for Counterfactual Identification and Estimation". *ICLR*. https://causalai.net/r87.pdf

[3] Plečo, D. & Bareinboim, E. (2024) "Causal Fairness Analysis", Foundations and Trends® in Machine Learning: Vol. 17, No. 3, pp 1–238. DOI: 10.1561/2200000106.https://causalai.net/r90.pdf

[4] Pearl, J. & Mackenzie, D.(2018) The Book of Why. Basic Books, New York, 2018.

[5] Bareinboim, E. (future?) "Causal Artificial Intelligence: A Roadmap for Building Causally Intelligent Systems". https://causalai-book.net/

[6] COMPAS dataset. https://github.com/propublica/compas-analysis